🛡️ DNS Rebinding and the NVIDIA App's Local MCP Server

Fulllion Creative Works

The Short Version

- **What it is:** NVIDIA App **11.0.9.565** (reported as an October 2026 beta) added an MCP server at `http://127.0.0.1:13508/mcp`, so AI assistants can do things like list your games, apply settings, and start ShadowPlay recordings.
- **Who can reach it:** Not the internet or your network directly, since it listens only on `127.0.0.1`. But your **web browser** and **any program on your PC** can.
- **The concern:** A malicious website can use **DNS rebinding** to make your browser send requests to that local server. Whether that works depends on whether the server checks the `Host` and `Origin` headers, and NVIDIA doesn't say.
- **What stops it:** Not CORS. **Rejecting unexpected `Host` and `Origin` headers** stops the browser route, and **authentication** stops other local programs.
- **Status:** No CVE or exploit exists for the NVIDIA App's server. The same class of bug has real CVEs in other MCP software, including one in another NVIDIA tool.
- **What to do:** If you don't use AI agents with the NVIDIA App, turn the feature off (Section 8).

Contents

1. What NVIDIA Added
2. Confirmed, Reported, and Unknown
3. Who Can Reach a Local Server
4. How a DNS Rebinding Attack Works
5. Why CORS Doesn't Stop It
6. How HTTP Header Validation Stops It
7. Real CVEs in the Same Class
8. Checking Your Own PC
9. Claims Worth Correcting
10. Takeaways
11. Sources

1. What NVIDIA Added

MCP is an open protocol that lets AI assistants call "tools" exposed by other software. NVIDIA's release notes for NVIDIA App **11.0.9.565** list "AI Agent Support for Model Context Protocol (MCP) server". They say a compatible assistant can see your installed games, apply optimal settings, or enable ShadowPlay to record gameplay. NVIDIA's blog of **October 13, 2026** announced MCP support for the NVIDIA App alongside a new NVIDIA App beta, and reporters tie that beta to 11.0.9.565.

NVIDIA's own reference on GitHub gives the technical details:

Detail	Value
Default address	`127.0.0.1`, port `13508`
Endpoint	`http://127.0.0.1:13508/mcp`
Fallback ports	`13509` to `13518`, then `49152` to `49161`
Discovery file	`%LOCALAPPDATA%\NVIDIA Corporation\NvAppMcpServer\server.json`
Other way in	A stdio bridge (`NvAppMcpServer.exe`) that relays to a per-user named pipe
On/off	A "master toggle" in the app's settings

Reporters and users list more actions than NVIDIA's release notes do, such as screenshots, instant replays, the performance overlay, launching games and apps, and reading driver versions. Treat those as reported, not documented.

> **Observed:** On a PC running the stable NVIDIA App **11.0.8.299** in November 2026, nothing was listening on port 13508. That fits the feature being limited to the 11.0.9.565 beta at the time of writing.

---

2. Confirmed, Reported, and Unknown

Claim	Status	Source
Added in NVIDIA App 11.0.9.565	**Confirmed**	NVIDIA release notes
Released as a beta on October 13, 2026	**Reported**, and consistent with NVIDIA's blog, which doesn't name the version	NVIDIA blog, news sites
Listens on `127.0.0.1:13508/mcp`	**Confirmed**	NVIDIA's GitHub reference
Can see games, apply settings, start ShadowPlay	**Confirmed**	NVIDIA release notes
Screenshots, overlay, launching apps, driver info	**Reported**	News sites and user posts
Switched on by default after updating	**Reported**	User reports; NVIDIA doesn't say

Claim	Status	Source
Requires no authentication	**Unknown**	NVIDIA's reference doesn't mention any; one report mentions a token
Checks the `Host` or `Origin` header	**Unknown**	Not documented
Has a CVE or known exploit	**None found**	No CVE, advisory or exploit as of November 2026

The setting's name differs between sources. NVIDIA's release notes place it under **Settings ▸ Features ▸ AI Agents and App Connections**. Reports describe a toggle named **Allow AI agents to control NVIDIA App**.

---

3. Who Can Reach a Local Server

A server bound to `127.0.0.1` (the "loopback" address) only accepts connections from the same computer. That rules out attackers on the internet or your Wi-Fi connecting to it directly. It still leaves two ways in:

Who	How	What stops it
A website you visit	Your browser runs its JavaScript on your computer, so the browser can connect to `127.0.0.1`. DNS rebinding (Section 4) is how a site gets around the browser's own protections	Header validation (Section 6), and newer browser protections
Another program on your PC	Any program can connect to `127.0.0.1`, and it can send whatever headers it likes	Authentication, such as a token only the intended client can read

Neither route gives an attacker more than the server itself offers. Here, that would mean the actions the MCP server exposes: seeing what's installed, changing settings, recording, and possibly launching software.

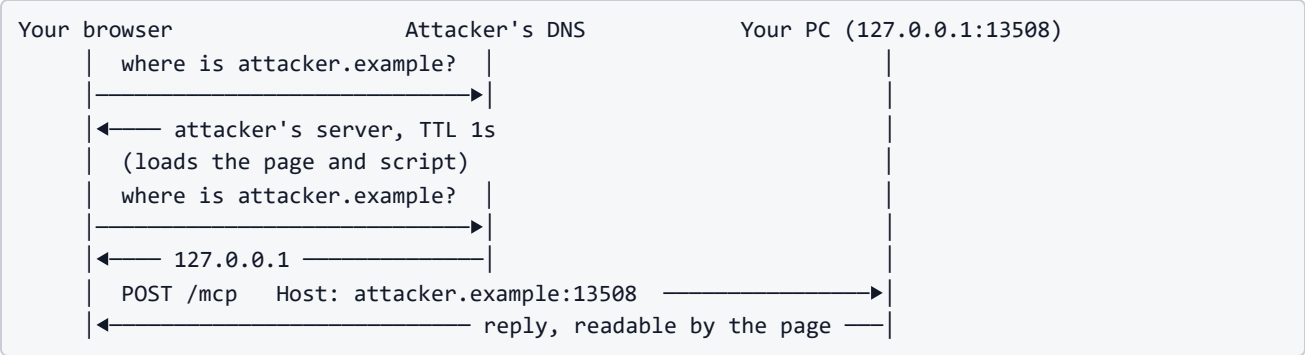
---

4. How a DNS Rebinding Attack Works

Browsers protect you with the **same-origin policy**: a page from `attacker.example` can't read responses from a different site. An "origin" is the scheme, host **name** and port, such as `http://attacker.example:13508`. The browser compares names, not IP addresses, and DNS rebinding exploits exactly that.

1. **You open the attacker's page** at `http://attacker.example:13508`. Its DNS answer points to the attacker's real server, with a very short time-to-live (TTL), so the browser soon forgets it.
2. **The page loads a script** and waits for that DNS answer to expire.
3. **The attacker changes the DNS answer** for `attacker.example` to `127.0.0.1`.

4. **The script sends a request** to `http://attacker.example:13508/mcp`. The browser looks up the name again, gets `127.0.0.1`, and connects to **your own computer**, on port 13508.
5. **The browser allows it, and lets the script read the reply.** As far as the browser knows, the page is still talking to its own origin, `http://attacker.example:13508`.
6. **The local server receives the request.** If it doesn't check where the request was meant to go, it treats it like any other client and runs the requested tool.



Two details are easy to miss:

- **The port has to match.** The page must be served on port 13508 too, because the port is part of the origin.
- **It isn't instant.** Browsers cache DNS answers, so real attacks keep retrying until the switch takes. Public toolkits exist to automate this, such as NCC Group's Singularity of Origin.

Newer browser protection: Chrome's **Local Network Access** asks your permission before a public website can reach local addresses, including loopback. Chrome's blog says the prompt launches in **Chrome 142**. It judges a request by the IP address it actually connects to, which is what makes it relevant to rebinding. Protection in other browsers varies, so a server can't count on it.

---

5. Why CORS Doesn't Stop It

A common claim is that browser **CORS** checks block websites from talking to local servers. That's wrong here, for two reasons.

1. **After rebinding, there's nothing cross-origin to check.** CORS controls requests from one origin to **another**. In step 4 above, the page at `http://attacker.example:13508` sends a request to `http://attacker.example:13508`. That's the same origin, so the browser sends no CORS preflight and lets the page read the reply.
2. **Even without rebinding, CORS doesn't block sending.** A "simple" request (a `GET`, or a `POST` with a form or plain-text body) gets no preflight at all. The browser still sends it. CORS only stops the page **reading the reply**, and for a request that triggers an action, sending it is the whole attack. The Local Network Access specification makes this point directly.

CORS does matter in one narrow case. A page that sends a `POST` with a JSON body straight to `http://127.0.0.1:13508`, with no rebinding, triggers a preflight that the server won't approve, so the browser never sends it. That's why attackers use rebinding.

---

6. How HTTP Header Validation Stops It

The defense has to live in the **server**, because the server is the only thing that sees every request. Two request headers give it what it needs, and the browser can't be made to fake either one.

6.1 The `Host` header

Every HTTP/1.1 request carries a `Host` header naming the site the client meant to reach. In a rebinding attack, the browser connects to `127.0.0.1`, but it still believes it's talking to `attacker.example`, so it sends:

```
POST /mcp HTTP/1.1
Host: attacker.example:13508
```

A real local client, by contrast, sends one of these:

```
Host: 127.0.0.1:13508
Host: localhost:13508
Host: [::1]:13508
```

So the server keeps an **allowlist** of the names it answers to, and refuses everything else:

- **Scripts can't change it.** `Host` is a forbidden request header, which a web page's JavaScript is not allowed to set.
- **It covers every request type.** Unlike `Origin` (below), the browser sends `Host` on every request, including `GET` and WebSocket upgrades.
- **This is the check that ends rebinding.** The attacker can point a name at `127.0.0.1`, but can't make the browser send a name the server accepts.

6.2 The `Origin` header

The `Origin` header names the page that **started** a request. Browsers send it on every cross-origin request, and on same-origin requests other than `GET` and `HEAD`. After rebinding, an MCP tool call (a `POST`) arrives with:

```
POST /mcp HTTP/1.1
Host: attacker.example:13508
Origin: http://attacker.example:13508
```

The MCP specification requires servers using its HTTP transport to validate `Origin` on every incoming connection to prevent DNS rebinding, and to answer **403 Forbidden** when it's present and invalid.

Two limits, which are why `Origin` works best alongside `Host`:

- **It's missing on same-origin `GET` requests,** so a check that only fires when `Origin` is present lets those through.
- **Non-browser clients often don't send it,** so a server can't simply require it, or it would break real MCP clients.

6.3 Putting it together

A minimal check, in Node.js:

```
const ALLOWED_HOSTS = new Set(['127.0.0.1:13508', 'localhost:13508', '[::1]:13508']);
const ALLOWED_ORIGINS = new Set(['http://127.0.0.1:13508', 'http://localhost:13508']);

function allowRequest(req, res) {
  const host = (req.headers.host || '').toLowerCase();
  if (!ALLOWED_HOSTS.has(host)) {
    res.writeHead(403).end('Forbidden: unexpected Host');
    return false;
  }
  const origin = req.headers.origin;
  if (origin !== undefined && !ALLOWED_ORIGINS.has(origin.toLowerCase())) {
    res.writeHead(403).end('Forbidden: unexpected Origin');
    return false;
  }
  return true;
}
```

How that check handles each case:

Request	`Host`	`Origin`	Result
Real MCP client on the same PC	`127.0.0.1:13508`	none	**Allowed**
Rebinding attack, tool call (`POST`)	`attacker.example:13508`	`http://attacker.example:13508`	**Blocked** by `Host`
Rebinding attack, `GET`	`attacker.example:13508`	none	**Blocked** by `Host`
A website calling `127.0.0.1` directly	`127.0.0.1:13508`	`https://attacker.example`	**Blocked** by `Origin`
A sandboxed or `file://` page	`127.0.0.1:13508`	`null`	**Blocked** by `Origin`

Getting it right:

- **Never allow `*`.** A wildcard turns the check off. CVE-2026-11624 (Section 7) shipped exactly that default.
- **Include the port,** and every name real clients use, including `localhost` and `[::1]`.
- **Compare exactly.** Don't use "contains" or "starts with" checks, which `127.0.0.1.attacker.example` would pass.

- **Bind to `127.0.0.1`, not `0.0.0.0`.** Header checks protect the browser route, not a server that's open to your whole network.

6.4 What header validation doesn't stop

Headers come from the client, and only browsers are prevented from faking them. **Any other program on your PC can send `Host: 127.0.0.1:13508`**, and the check above will let it in. That needs a different defense:

- **Authentication:** a secret token that only the intended client can read, sent with every request. The MCP specification recommends authentication for all connections, and suggests a token or an access-controlled channel for local servers.
- **Operating-system access control:** a channel the operating system can limit to one user, such as a per-user named pipe. NVIDIA's stdio bridge uses one, though the HTTP endpoint is still there alongside it.

Defense	Stops rebinding from a website	Stops other programs on your PC
Binding to `127.0.0.1`	No	No
CORS	No (Section 5)	No
Browser Local Network Access	Yes, in browsers that have it and if you deny the prompt	No
`Host` allowlist	**Yes**	No
`Origin` check	Yes, for requests that carry it	No
Authentication token	Yes	**Yes**, unless the program can read the token
Turning the feature off	**Yes**	**Yes**

---

7. Real CVEs in the Same Class

None of these affect the NVIDIA App. They show the pattern is real, and how it has been fixed.

CVE	Product	What NVD says	Severity	Fix
CVE-2025-66416	MCP Python SDK (`mcp` on PyPI)	Before 1.23.0, DNS rebinding protection wasn't on by default for HTTP servers, so a malicious website could invoke a local server's tools	High (GitHub: CVSS 4.0, 7.6)	Upgrade to 1.23.0
CVE-2026-11624	Google MCP Toolbox for Databases	Before 0.25.0, there was no way to validate the host. 0.25.0 added `--allowed-hosts` beside `--allowed-origins`, but both default to `*` and only print a warning	Not yet scored by NVD	Upgrade to 0.25.0 **and** set both flags

CVE	Product	What NVD says	Severity	Fix
CVE-2026-65105	NVIDIA NemoClaw for Linux, up to 0.0.25	Its inference server could be reached without authentication	High (NVIDIA: CVSS 3.1, 8.1)	Follow NVIDIA's advisory

On NemoClaw, the Cloud Security Alliance's analysis explains how a single malicious page could reach that unauthenticated local backend through DNS rebinding. The root flaw is the missing authentication (CWE-306). Rebinding is how a website reaches it.

---

8. Checking Your Own PC

8.1 Is the server running?

In PowerShell, list anything listening on the ports NVIDIA's reference names:

```
$ports = @(13508..13518) + @(49152..49161)
Get-NetTCPConnection -State Listen -ErrorAction SilentlyContinue |
  Where-Object LocalPort -in $ports |
  Select-Object LocalAddress, LocalPort, @{n='Process'; e={(Get-Process -Id
$_.OwningProcess).ProcessName}}
```

No output means nothing is listening on those ports. Other software can use the `49152` range too, so check the **Process** column before drawing conclusions.

You can also check for the discovery file:

```
Test-Path "$env:LOCALAPPDATA\NVIDIA Corporation\NvAppMcpServer\server.json"
```

8.2 Turn it off

In the NVIDIA App, go to **Settings ▸ Features**, find the AI agents section (**AI Agents and App Connections** in NVIDIA's release notes), and switch off **Allow AI agents to control NVIDIA App**. Then run the check in 8.1 again to confirm nothing is listening.

8.3 Optional: does it check the `Host` header?

If you leave the feature on, you can see how your own copy responds to an unexpected `Host`, the way a rebinding attack would arrive. Compare the two responses:

```
curl.exe -i --max-time 5 http://127.0.0.1:13508/mcp
curl.exe -i --max-time 5 -H "Host: attacker.example:13508" http://127.0.0.1:13508/mcp
```

`--max-time 5` stops each request after 5 seconds, since an MCP server may hold a `GET` open as a stream.

- **The second is refused (such as `403`) and the first isn't:** the server rejects unexpected hosts.

- **Both get the same response:** inconclusive. The server may not check `Host`, or it may check later, or require authentication first.

Only test software on your own computer.

---

9. Claims Worth Correcting

Answers circulating about this feature, including AI-generated ones, contain several errors:

Claim	Correction
"CORS preflight checks block websites from sending commands to local ports"	Not after DNS rebinding, which makes the request same-origin, and not for simple requests either (Section 5)
"The server is unauthenticated"	Not established. NVIDIA doesn't document authentication either way, and one report mentions a token
CVE-2026-11624 is a flaw in "the core MCP SDKs"	NVD lists it against **Google's MCP Toolbox for Databases**
CVE-2026-65105 is a "DNS rebinding flaw" in NemoClaw	NVD classes it as **missing authentication** (CWE-306). Rebinding is how a website reaches it
"Turning it off is basic hygiene because it's actively exploited"	No exploit or CVE exists for the NVIDIA App's server. Turning it off is still sensible if you don't use it

---

10. Takeaways

For users

- Turn off local AI-agent servers you don't use. A closed port can't be attacked.
- Keep your browser updated, and answer **Block** when a website asks for access to your local network.

For developers of local servers

- Bind to `127.0.0.1`, never `0.0.0.0`.
- Validate `Host` against an exact allowlist, and validate `Origin` whenever it's present. Never default either to `*`.
- Require authentication, even on localhost, because header checks don't stop other local programs.
- Ship new agent-control features switched off, and say so in the release notes.

---

11. Sources

NVIDIA

- NVIDIA App release highlights (11.0.9.565)
- NVIDIA App MCP connection reference (GitHub)
- NVIDIA blog, October 13, 2026

Reporting

- Let's Data Science: NVIDIA adds local MCP control to NVIDIA App
- Windows Mode: NVIDIA App AI agents and how to turn them off
- Cyberiack: How to shut off the AI agent access

Specifications and browser behavior

- MCP specification: transports and security warning
- MCP specification: security best practices
- Chrome: Local Network Access
- Local Network Access specification (WICG)
- MDN: Origin header
- MDN: Cross-Origin Resource Sharing (CORS)
- MDN: Same-origin policy

Vulnerabilities

- NVD: CVE-2025-66416 (MCP Python SDK)
- NVD: CVE-2026-11624 (Google MCP Toolbox for Databases)
- NVD: CVE-2026-65105 (NVIDIA NemoClaw)
- Cloud Security Alliance: NemoClaw DNS rebinding research note

Tools

- NCC Group: Singularity of Origin (DNS rebinding toolkit)

Latest version: github.com/ImmuneMoon/NVIDIA-App-MCP-DNS-Rebinding-Writeup

If this helped, you can support Fulllion Creative Works through PayPal or Ko-fi.

© 2026 Fulllion Creative Works. All rights reserved.